APPENDIX
PROOFS

A. Proof of Proposition 1

Proof. We prove both inclusions.

First, let q be a domain-independent series calculus query. Its local active domains are admissible, so domain independence gives

$$q_{\mathbf{d}}(\mathbf{I}) = q_{\mathbf{d}_{adom}}(\mathbf{I})$$

for every input $\mathbf{I}$ and admissible $\mathbf{d}$. Thus the same query defines the same mapping under active-domain evaluation, proving

$$CALC_{di} \sqsubseteq CALC_{adom}.$$

For the converse, let q be a series calculus query with formula $\Pi = \{\gamma, \varphi\}$, evaluated on its local active domains. We construct a domain-independent query q^* defining the same mapping.

Fix

$$C = \mathrm{adom}(\varphi), \qquad \mathbf{d}_0[t] = \mathrm{adom}(\mathbf{I}[t]) \cup C.$$

Here, C is always the constant set of the original formula.

For each input schema $S = p_S(T, A_1, \ldots, A_{n_S})$, let $\mathbf{z}_S = (z_1, \ldots, z_{n_S})$ consist of fresh variables. Define the formula abbreviation

$$A_C(T,x) = \bigvee_{c \in C} (x = c) \vee \bigvee_{S \in \mathbf{S}} \bigvee_{j=1}^{n_S} \exists \mathbf{z}_S \left(S(T, \mathbf{z}_S) \wedge x = z_j \wedge x \neq \bot \right).$$

The first disjunction recognizes the original query constants. The second recognizes non-absent values occurring in input records at T. All auxiliary variables are fresh at each occurrence. Since the input schema collection is finite, A_C is a finite formula in the existing calculus.

We will include the conjunct $\bot = \bot$ in the new query. Hence every domain admissible for that query contains $\bot$, allowing the auxiliary variables to match complete source records even when some fields are absent. For every such domain $\mathbf{d}$ and every $a \in \mathbf{d}[t]$,

$$\mathbf{I}, [T \mapsto t, x \mapsto a] \models_{\mathbf{d}} A_C(T,x) \quad \Longleftrightarrow \quad a \in \mathbf{d}_0[t]. \tag{1}$$

Indeed, every non-absent input value belongs to $\mathbf{d}[t]$, and the remaining fields of its source record are either input values or $\bot$. Conversely, a satisfied source branch supplies an actual non-absent observation, while a satisfied constant branch supplies a member of C. If $\bot \notin C$, the formula A_C excludes $\bot$ despite its presence in the enlarged candidate domain.

Next define a transformation R on the subformulas of the original φ. Source atoms, value comparisons, timestamp predicates, and source-position predicates remain unchanged. For Boolean connectives, let

$$R(\psi \wedge \xi) = R(\psi) \wedge R(\xi),$$
$$R(\psi \vee \xi) = R(\psi) \vee R(\xi),$$
$$R(\neg\psi) = \neg R(\psi).$$

For quantified formulas, let

$$R(\exists x\, \psi) = \exists x \left(A_C(T,x) \wedge R(\psi) \right),$$
$$R(\forall x\, \psi) = \forall x \left(\neg A_C(T,x) \vee R(\psi) \right).$$

These clauses transform only the original formula; the auxiliary quantifiers introduced inside A_C are not transformed again.

Let F be the set of all free value variables of φ. Construct

$$\varphi^* = (\bot = \bot) \wedge \bigwedge_{x \in F} A_C(T,x) \wedge R(\varphi).$$

Let q^* retain the index formula γ, output variables, and declared output schema of q, replacing φ with φ^*. The guards cover all free value variables, including those not appearing in the output.

Let $\mathbf{d}$ be any domain family admissible for q^*. Its local domains contain all input observations, the constants in C, and $\bot$. By structural induction on each original subformula ψ, for every valuation ν with $\nu(T) = t$ and free value variables of ψ assigned from $\mathbf{d}_0[t]$, we have

$$\mathbf{I}, \nu \models_{\mathbf{d}} R(\psi) \quad \Longleftrightarrow \quad \mathbf{I}, \nu \models_{\mathbf{d}_0} \psi. \tag{2}$$

The atomic cases use the same input records and assigned values. The Boolean cases follow from the induction hypothesis. For the quantified cases, (1) restricts the quantified values to exactly $\mathbf{d}_0[t]$, so the existential and universal conditions agree with the original active-domain evaluation.

The outer guards in φ^* restrict every free value variable to that same domain. Moreover, γ uses only timestamp and source-position conditions, whose interpretation is unchanged. Consequently, (2) gives

$$q^*_{\mathbf{d}}(\mathbf{I}) = q_{\mathbf{d}_0}(\mathbf{I})$$

for every input $\mathbf{I}$ and every domain $\mathbf{d}$ admissible for q^*.

The right-hand side is the original active-domain result. Therefore q^* preserves the declared schema, instance constraints, and output-observation requirements of q. It is thus an admitted core series calculus query.

For any two admissible domain families $\mathbf{d}$ and $\mathbf{d}'$ for q^*,

$$q^*_{\mathbf{d}}(\mathbf{I}) = q_{\mathbf{d}_0}(\mathbf{I}) = q^*_{\mathbf{d}'}(\mathbf{I}).$$

Hence q^* is domain-independent and defines the same mapping as q under active-domain evaluation. This proves

$$CALC_{adom} \sqsubseteq CALC_{di}.$$

Combining the two inclusions yields

$$CALC_{di} \equiv CALC_{adom}.$$

□

B. Proof of Lemma 2

Proof. We construct an equivalent calculus query by structural induction on e.

For each subexpression g, let S_g be its output schema and V_g its set of full value-attribute paths. Associate a distinct variable u_A with each $A \in V_g$. Variable vectors follow the attribute order of their schemas. All bound variables are renamed when necessary to avoid capture.

We construct a formula $\Phi_g(T, \mathbf{u}_{V_g})$ such that, for every admissible $\mathbf{d}$, every $t \in \mathrm{adom}_T(\mathbf{I})$, and every value vector $\mathbf{a}$ from $\mathbf{d}[t]$,

$$\mathbf{I}, [T \mapsto t, \mathbf{u}_{V_g} \mapsto \mathbf{a}] \models_{\mathbf{d}} \Phi_g \quad \Longleftrightarrow \quad \langle t, \mathbf{a} \rangle \in g(\mathbf{I}). \qquad (*)$$

Every core operator retains existing timestamps. Moreover, every non-absent output value at t comes from an input value attribute at t. Selection, Split, and Difference retain existing values; Alignment combines them and may insert $\perp$; Concatenation chooses a complete input record; Renaming changes only paths. These observations apply inductively to all subexpressions.

We retain the constant $\perp$ in every constructed formula. Thus every admissible local domain contains both the observed input values and $\perp$, making every intermediate result tuple available as a valuation.

For a constructed formula, write

$$E_g(T) = \exists \mathbf{v}\, \Phi_g(T, \mathbf{v}),$$

where $\mathbf{v}$ consists of fresh variables for all value attributes of g. By (*) and the availability of result values, $E_g(t)$ holds exactly when $g(\mathbf{I})$ contains a record at t.

Input reference. For $g = S = p(T, A_1, \ldots, A_n)$, let

$$\Phi_S = S(T, u_{p.A_1}, \ldots, u_{p.A_n}) \wedge (\perp = \perp).$$

The final conjunct is true under the defined equality semantics and ensures that $\perp$ belongs to the formula's constant set. Property (*) follows from the semantics of source atoms.

Selection. For $g = \sigma_\theta(g_1)$, let

$$\Phi_g = \Phi_{g_1} \wedge \widehat{\theta},$$

where $\widehat{\theta}$ replaces each attribute reference A with u_A and retains timestamp comparisons. If θ contains a position predicate, the lemma requires $g_1 = S$ for an input reference S; replace its position variable by i_S. The formula and the operator therefore test the same conditions on the same input records. By induction, (*) holds for Selection.

Split. For $g = \psi_B(g_1)$, set

$$\Phi_g(T, \mathbf{u}_B) = \left(\exists \mathbf{u}_{V_{g_1} \setminus B}\, \Phi_{g_1}(T, \mathbf{u}_{V_{g_1}})\right) \wedge \bigvee_{A \in B} (u_A \neq \perp).$$

The existential quantifiers retain precisely the selected fields of input records. Their witnesses belong to every admissible local domain. The final condition excludes records whose selected fields are all absent. The output schema retains the paths in B under their canonical prefix, exactly as specified by Split.

Alignment. For $g = g_1 \mapsto g_2$, let V_1, V_2 be the disjoint full value-attribute sets of the two inputs. Set

$$\begin{aligned}\Phi_g = {} & (\Phi_{g_1} \wedge \Phi_{g_2}) \\ & \vee \left(\Phi_{g_1} \wedge \neg E_{g_2}(T) \wedge \bigwedge_{A \in V_2} (u_A = \perp)\right) \\ & \vee \left(\Phi_{g_2} \wedge \neg E_{g_1}(T) \wedge \bigwedge_{A \in V_1} (u_A = \perp)\right).\end{aligned}$$

The three branches respectively handle timestamps present in both inputs, only in the left input, and only in the right input. The latter two branches fill the absent-side fields with $\perp$. By induction, these branches produce exactly the records specified by Alignment. The output uses $V_1 \cup V_2$ under its canonical prefix.

Concatenation. For $g = g_1 \triangleright g_2$, let $h : V_1 \to V_2$ be the attribute correspondence required by the operator. Write $\Phi^h_{g_2}$ for the formula obtained by replacing each free output variable corresponding to $h(A)$ with u_A. This substitution represents the right records under the left schema.

Set

$$\Phi_g = \Phi_{g_1} \vee \left(\Phi^h_{g_2} \wedge \neg E_{g_1}(T)\right).$$

When the left input contains a record at T, only that complete record is retained, including its absent fields. Otherwise, the right record is retained under the left schema. This is exactly the definition of Concatenation.

Difference. For $g = g_1 - g_2$, use the same attribute correspondence and let

$$\Phi_g = \Phi_{g_1} \wedge \neg \Phi^h_{g_2}.$$

Under a fixed assignment to the left output variables, $\Phi^h_{g_2}$ holds exactly when the corresponding complete record occurs in the right input. The formula therefore removes precisely the matching records, including matching absent fields, and retains the left schema. Hence (*) holds for Difference.

Renaming. For $g = \rho_f(g_1)$, replace each free output variable u_A in Φ_{g_1} with $u_{f(A)}$. Keep input source names, timestamp conditions, and source-position references unchanged. Declare the output schema using the mapped full paths under their canonical prefix.

Since f is injective on the affected attributes, this substitution preserves the timestamp and values of each record and changes only its output paths. Thus (*) holds for Renaming.

All constructed formulas belong to the recursive syntax. Each formula retains the constant $\perp$. Furthermore, its constants include those of its constituent formulas, so a domain admissible for a parent formula is also admissible for its children. The induction therefore establishes (*) under every admissible domain family.

Let the output schema of e be $S_e = p'(T, B_1, \ldots, B_m)$. Construct the named query

$$q = \left\{\left(T, (u_{p'.B_1}, \ldots, u_{p'.B_m} : B_1, \ldots, B_m)\right) \mid \Pi\right\},$$

where

$$\Pi = \{T \in \mathrm{adom}_T(\mathbf{I}), \Phi_e\}.$$

All timestamps and values of $e(\mathbf{I})$ are available to this query. Property (*) consequently gives

$$q_{\mathbf{d}}(\mathbf{I}) = e(\mathbf{I})$$

with the same output schema.

The result is a valid time-series instance. Selection and Difference retain subsets of valid instances; Split removes all-absent selected records; Alignment and Concatenation produce at most one nonempty record per timestamp; and Renaming preserves the instance constraints. Empty results retain their declared schemas.

Finally, for any two admissible domain families $\mathbf{d}$ and $\mathbf{d}'$,

$$q_{\mathbf{d}}(\mathbf{I}) = e(\mathbf{I}) = q_{\mathbf{d}'}(\mathbf{I}).$$

Hence q is domain-independent. □

C. Proof of Lemma 3

Proof. Let q have formula $\Pi = \{\gamma, \varphi\}$, output variables $e_1, \ldots, e_m$, and output schema

$$S' = p'(T, B_1, \ldots, B_m).$$

Write $b_j = p'.B_j$ for its full output paths. We construct an expression using the six core operators.

Aligned input. Let $S_1, \ldots, S_r$ be the input schemas. Give their fields distinct paths by renaming existing nodes, and align the resulting instances. These aliases do not insert additional path levels. Write a_{ij} for the aliased field corresponding to the jth value attribute of S_i.

For every source-position predicate β occurring in γ or φ, first apply its position test directly to its source instance. Give this filtered copy fresh paths and include it in the alignment. Let H denote the resulting expression.

For each input $\mathbf{I}$, $H(\mathbf{I})$ contains exactly one record at every timestamp in $\mathrm{adom}_T(\mathbf{I})$. It contains the original observations and the filtered copies, with absent-side fields filled by $\perp$.

Define the predicate

$$P_i = \bigvee_j (a_{ij} \neq \perp).$$

Since an input record contains at least one non-absent value, P_i holds exactly when S_i has a record at the current timestamp. Likewise, testing whether any field of the filtered copy for β is non-absent gives a predicate P_β that holds exactly when β holds. These are abbreviations for Selection predicates.

Candidate values. Let

$$C = \mathrm{adom}(\varphi)$$

be the value constants of the original query. Form the finite list of terms

$$L = \{a_{ij} \mid 1 \leq i \leq r\} \cup C,$$

where all value attributes of every input schema are included. For each candidate term ℓ, define its availability condition by

$$G_\ell = \begin{cases} a_{ij} \neq \perp, & \ell = a_{ij}, \\ \mathsf{true}, & \ell \in C. \end{cases}$$

At the record of $H(\mathbf{I})$ with timestamp t, the available terms evaluate to exactly

$$\mathbf{d}_{adom}[t] = \mathrm{adom}(\mathbf{I}[t]) \cup C.$$

Different terms may denote the same value; such repetitions do not affect existential or universal quantification. The constant set C remains fixed throughout the construction. In particular, absence tests introduced by the translation do not enlarge the original candidate domain.

Formula translation. For a subformula ψ and an assignment η of its free value variables to candidate terms, construct a Selection predicate Θ^η_ψ over H. Extend η to constants by $\eta(c) = c$. Bound variables are renamed to avoid capture.

For a source atom, set

$$\Theta^\eta_{S_i(T,u_1,\ldots,u_n)} = P_i \wedge \bigwedge_{j=1}^{n} \big(a_{ij} = \eta(u_j)\big).$$

Thus the corresponding source record must exist and its fields must match the assigned values.

For a value comparison, substitute its terms:

$$\Theta^\eta_{\tau_1 \phi \tau_2} = \eta(\tau_1) \phi \eta(\tau_2).$$

Timestamp comparisons are retained, and each source-position predicate β is replaced by P_β. All comparisons use the calculus semantics.

Boolean connectives are translated recursively:

$$\begin{aligned} \Theta^\eta_{\psi \wedge \xi} &= \Theta^\eta_\psi \wedge \Theta^\eta_\xi, \\ \Theta^\eta_{\psi \vee \xi} &= \Theta^\eta_\psi \vee \Theta^\eta_\xi, \\ \Theta^\eta_{\neg \psi} &= \neg \Theta^\eta_\psi. \end{aligned}$$

For quantified variables, set

$$\begin{aligned} \Theta^\eta_{\exists x\, \psi} &= \bigvee_{\ell \in L} \left(G_\ell \wedge \Theta^{\eta[x \mapsto \ell]}_\psi \right), \\ \Theta^\eta_{\forall x\, \psi} &= \bigwedge_{\ell \in L} \left(\neg G_\ell \vee \Theta^{\eta[x \mapsto \ell]}_\psi \right). \end{aligned}$$

For every assignment η whose terms are available at t, structural induction on ψ shows that Θ^η_ψ holds on the aligned record exactly when ψ holds under the corresponding valuation. The atomic cases follow from source membership and comparison semantics. The Boolean cases follow immediately from their definitions. The quantified cases follow because the available terms enumerate exactly the original local active domain.

Consequently, the translation produces a finite Boolean Selection predicate without introducing a relation containing multiple candidate records at the same timestamp.

Output assignments. For each output position j, choose either an input-field term a_{ik} whose domain equals $\mathrm{Dom}(b_j)$, or the term $\perp$ if $\perp \in C$. Let

$$\alpha = (\alpha_1, \ldots, \alpha_m)$$

be such a choice. There are finitely many choices.

Define

$$F_\alpha = \bigwedge_{j=1}^{m} G_{\alpha_j} \wedge \bigwedge_{\substack{1 \le j < k \le m \\ e_j = e_k}} (\alpha_j = \alpha_k).$$

The second conjunction ensures that repeated output variables receive the same value. Let η_α assign each distinct output variable the term chosen for its first occurrence.

Let $\mathbf{y}$ list the free value variables of φ that are not output variables. Their assignments are existentially quantified when constructing the output tuple. Replace source-position tests in γ as above and express its timestamp restriction by Selection, obtaining $\widehat{\gamma}$. Set

$$\theta_\alpha = \widehat{\gamma} \wedge F_\alpha \wedge \Theta^{\eta_\alpha}_{\exists \mathbf{y}\, \varphi}, \qquad H_\alpha = \sigma_{\theta_\alpha}(H).$$

Thus H_α contains exactly the timestamps at which the values selected by α form a result tuple of q.

Choices assigning $\perp$ to every output position may be omitted, because a valid time-series query cannot return an all-absent record.

Output construction. Because S' is type-derivable, choose an expression $K : S'$. Its values are irrelevant: use a false Selection predicate to obtain an empty instance of the required schema,

$$Z = \sigma_{\mathsf{false}}(K).$$

Here false can be expressed as $b_1 \neq b_1$. Choosing K uses only the typing derivation; it does not assume an expression already equivalent to q.

For a non-absent output position j with $\alpha_j = a_{ik}$, extract

$$\psi_{\{a_{ik}\}}(H_\alpha).$$

Its selected value is non-absent at every timestamp of H_α, by F_α. Rename its leaf to the leaf name of b_j and denote the resulting expression by $R_{\alpha,j}$. Construct

$$E_{\alpha,j} = \psi_{\{b_j\}}(Z) \triangleright R_{\alpha,j}.$$

The left input is empty and declares the target single-field schema. The two inputs have matching leaf names and equal value domains, so Concatenation is legal and places the selected values under the target path b_j.

Align the expressions $E_{\alpha,j}$ for all non-absent output positions. Their paths are distinct, and their timestamp domains all equal that of H_α. If M_α is the nonempty set of output paths assigned $\perp$, also align the empty instance

$$\psi_{M_\alpha}(Z).$$

This fills precisely those remaining fields with $\perp$. Let E_α denote the resulting expression.

The construction gives E_α output schema S' and

$$E_\alpha(\mathbf{I}) = \{\langle t, \alpha_1(t), \ldots, \alpha_m(t)\rangle \mid t \in \mathrm{adom}_T(H_\alpha(\mathbf{I}))\}.$$

Here $\alpha_j(t)$ denotes the value of the selected candidate term at t. An empty branch retains the same declared schema.

Combining branches. Every tuple of $E_\alpha(\mathbf{I})$ satisfies the original query formula under active-domain evaluation. Hence

$$E_\alpha(\mathbf{I}) \subseteq q_{\mathbf{d}_{adom}}(\mathbf{I}).$$

Conversely, consider a result tuple of q. For each non-absent output field, choose a same-timestamp source attribute with the required domain, as stipulated in the lemma. For an absent output field, choose $\perp$. Such an absent value can occur in the active-domain valuation only if $\perp \in C$. These choices give an assignment α satisfying F_α. The original satisfying valuation supplies witnesses for the variables in $\mathbf{y}$. By the formula-translation induction, θ_α holds, so the tuple belongs to $E_\alpha(\mathbf{I})$. Therefore,

$$q_{\mathbf{d}_{adom}}(\mathbf{I}) = \bigcup_\alpha E_\alpha(\mathbf{I}).$$

All branches are subsets of the same valid time-series result. Consequently, whenever two branches contain a record at the same timestamp, their complete records are identical. Concatenation therefore implements their union without discarding any distinct result.

If S' contains repeated leaf names under different parents, apply the same finite sequence of node renamings ρ to every branch, giving all output leaves distinct fresh names. The renamed branches satisfy the input requirements of Concatenation. For the finitely many branches $\alpha_1, \ldots, \alpha_N$, let

$$e = \rho^{-1}\left(\rho(E_{\alpha_1}) \triangleright \cdots \triangleright \rho(E_{\alpha_N})\right),$$

where ρ^{-1} restores the original leaf names. Both ρ and ρ^{-1} abbreviate finite compositions of the defined Renaming operator. If no branch remains, let $e = Z$.

Every step uses only the six core operators, and all intermediate results satisfy their typing requirements. The construction depends only on the query and input schemas. For every input $\mathbf{I}$, it yields

$$e(\mathbf{I}) = q_{\mathbf{d}_{adom}}(\mathbf{I}),$$

with the declared output schema S'. □